```
namespace Some\Other\Sub\Space {
    use function \Other\Sub\collision as otherCollision;
    use function \collision as rootCollision;

    otherCollision();
    rootCollision();
}
```

```
Output:
Hello from Other\\Sub namespace.
Hello from root namespace.
```

Namespaces Are Not Scope

We explored the concept of scope earlier in the chapter and how it provided a mechanism for protecting variables and data inside a function from those outside: local vs. global scope. Namespaces are another way to prevent symbols in code from colliding with each other. Scope only concerns variables. A namespace concerns only functions, classes, interfaces, and constants. Changing namespaces does not change scope; only entering a function boundary can change scope. The following code should illustrate this:

```
namespace Other {
    $val = 'I am everywhere' . PHP_EOL;

    function greetings(): void
    {
        print "I am also everywhere, but did you get my name right?" .
        PHP_EOL;
    }
}

namespace {
    print $val;
    \Other\greetings();
    greetings();
}
```